Design and Implementation of a Real-Time Clock and Stopwatch using 8051 Microcontroller

Embedded Systems Laboratory Course Project

Samvab De
Roll No.: 22EC37031

Supervisor: Prof. Indrajit Chakrabarti
Department of Electronics and Electrical Communication Engineering
Indian Institute of Technology Kharagpur

1 Introduction

Embedded systems frequently require accurate timing, periodic event handling, and interfacing with external input/output peripherals. Microcontroller timers and interrupts provide an efficient mechanism for implementing such applications without requiring the main program to continuously execute software delay loops.

In this project, a real-time digital clock and stopwatch system was implemented using an 8051 microcontroller. The system was programmed using 8051 assembly language in Keil uVision and downloaded to the target microcontroller kit using WIN51E. Common-cathode 7-segment displays and control switches were interfaced with the system through the 8255 Programmable Peripheral Interface (PPI).

The system operates in two modes: a normal clock mode and a stopwatch mode. In clock mode, the current time is displayed in hours, minutes, and seconds. In stopwatch mode, the stopwatch timing operation is controlled through the external input switches. Timer interrupts and an Interrupt Service Routine (ISR) were used to continuously update the timing variables.

The implementation combines several fundamental embedded-system concepts, including assembly-level programming, peripheral interfacing, display multiplexing, timer configuration, interrupt handling, and real-time software control.

2 System Specifications

The major specifications and functional components of the implemented system are summarized in Table 1.

Table 1: System specifications.

Parameter	Specification
Microcontroller	8051 microcontroller
Programming Language	8051 Assembly Language
Development Environment	Keil uVision
Programming/Download Tool	WIN51E
Display Interface	Six common-cathode 7-segment digits
Peripheral Interface	8255 Programmable Peripheral Interface (PPI)
Operating Modes	Real-time clock and stopwatch
Display Format	HH-MM-SS
Display Technique	Six-digit multiplexed display
Timing Method	Timer 1 periodic interrupts
Timer Mode	Mode 1, 16-bit timer
Timer Reload Values	TH1 = 3CH, TL1 = B0H
Interrupt Handling	Timer interrupt and ISR
Clock Storage	External memory locations 9000H–9002H
Stopwatch Storage	External memory locations 9003H–9005H

3 System Architecture

The overall system architecture is shown conceptually in Figure 1. The 8051 microcontroller acts as the central processing unit and executes the assembly program responsible for timer configuration, interrupt handling, switch processing, and display control.

The external displays and control switches are interfaced through the 8255 Programmable Peripheral Interface. The output ports are used to transfer segment data and digit-selection signals to the display system, while the input port is used to monitor the control switches.

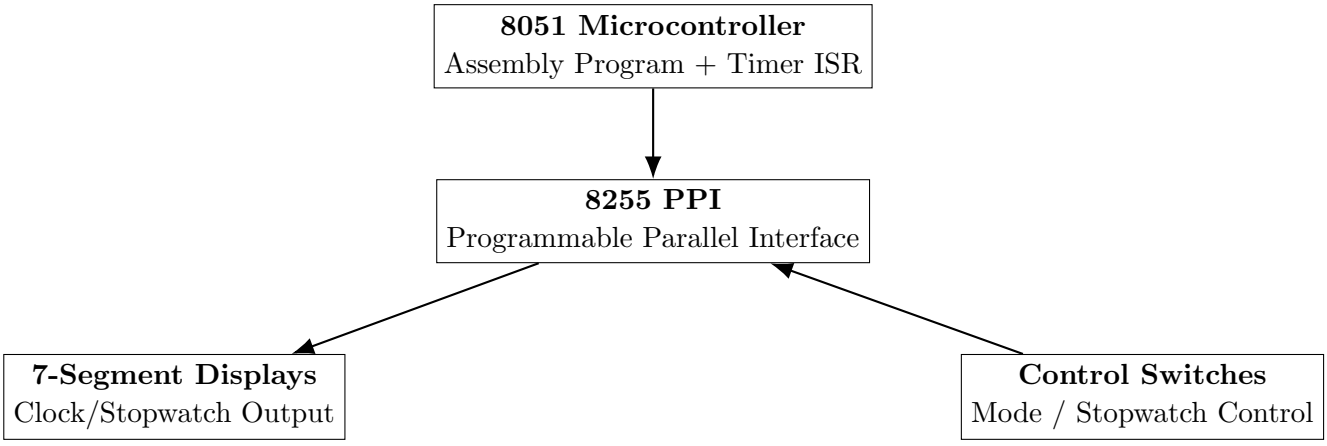


Figure 1: Block-level architecture of the real-time clock and stopwatch system.

The major functional blocks of the system are:

- **8051 Microcontroller:** Executes the assembly program and manages timer operation, interrupts, and system control.

- **Timer and Interrupt Unit:** Generates periodic timing events for updating the clock and stopwatch values.
- **Interrupt Service Routine:** Performs timer-based updating of the clock and conditionally updates the stopwatch.
- **8255 PPI:** Provides the parallel interface between the microcontroller, display modules, and control switches.
- **7-Segment Display Interface:** Displays the time information in HH-MM-SS format using multiplexed common-cathode displays.
- **Control Switches:** Used for mode selection and stopwatch control.

4 Memory Organization

Separate external memory locations were allocated for storing the clock and stopwatch timing variables. This allows the normal clock and stopwatch to be maintained independently within the same interrupt-driven timing system.

The memory organization used in the implementation is summarized in Table 2.

Table 2: Memory allocation used for timing and control variables.

Memory Location	**Stored Variable**
9000H	Clock hours
9001H	Clock minutes
9002H	Clock seconds
9003H	Stopwatch hours
9004H	Stopwatch minutes
9005H	Stopwatch seconds
9010H	Stopwatch active flag
9011H	Display mode flag

The display mode flag determines whether the clock values or stopwatch values are loaded into the display buffer. The stopwatch active flag determines whether the stopwatch timing routine is executed during the timer event.

5 8051 Assembly Program Development

The complete control program was developed using 8051 assembly language in Keil uVision. Assembly programming provides direct control over the microcontroller registers, timers, interrupt configuration, memory locations, and input/output ports.

The program was organized into several functional routines, including:

- Delay generation for display multiplexing.
- 7-segment display driver routines.
- Six-digit display multiplexing.

- Clock reset and stopwatch reset routines.
- Clock and stopwatch increment routines.
- Display mode handling.
- Timer interrupt service routine.
- Main program initialization and switch handling.

After development and assembly, the program was downloaded to the target microcontroller kit using WIN51E. The hardware implementation was verified by observing the displayed time values and the response of the system to mode and stopwatch control inputs.

6 8255 Peripheral and Display Interfacing

6.1 8255 Programmable Peripheral Interface

The 8255 Programmable Peripheral Interface was used to connect the external display modules and switches to the 8051 microcontroller. The programmable parallel ports provide the required input and output interface for the system.

In the implementation, the ports were used for the following functions:

- Transferring segment data to the 7-segment displays.
- Selecting the individual display digits during multiplexing.
- Reading the mode selection input.
- Reading the stopwatch control input.

The assembly program configures the 8255 during initialization before entering the main program loop.

6.2 Multiplexed 7-Segment Display Interface

The system uses six common-cathode 7-segment digits to display the time in HH-MM-SS format. The displays are operated using time-division multiplexing, where the segment data is shared while individual digits are activated sequentially.

A lookup table containing the segment patterns corresponding to decimal digits 0–9 is used by the display driver. The digit value is used as an offset into the lookup table to obtain the required segment pattern.

The display driver transfers the segment data to the output port and then activates the corresponding display digit using an active-low digit-select mask. A short delay is inserted before moving to the next digit.

The display refresh sequence is:

Hours Tens → Hours Units → Minutes Tens → Minutes Units → Seconds Tens → Seconds Units

The rapid repetition of this sequence creates the appearance of a continuously illuminated six-digit display.

Table 3: Digit selection masks used during display multiplexing.

Display Position	**Active-Low Selection Mask**
Hours Tens	11011111B
Hours Units	11101111B
Minutes Tens	11110111B
Minutes Units	11111011B
Seconds Tens	11111101B
Seconds Units	11111110B

7 Timer Interrupt and ISR Implementation

Accurate timekeeping was implemented using Timer 1 of the 8051 microcontroller. The timer was configured in Mode 1, which provides 16-bit timer operation.

The timer registers were initialized using the values:

$$TH1 = 3CH, \qquad TL1 = B0H$$

When the timer overflows, a Timer 1 interrupt is generated and program control is transferred to the corresponding Interrupt Service Routine (ISR).

A software counter is used to accumulate successive timer overflows and generate the required one-second timing event. The ISR reloads the timer, updates the software counter, and performs the required time-update operations when the timing interval is reached.

The general interrupt sequence is:

$$\text{Timer Overflow} \rightarrow \text{Interrupt Request} \rightarrow \text{ISR Execution} \rightarrow \text{Time Update}$$

The ISR performs the following operations:

1. Stops Timer 1 during interrupt processing.
2. Reloads the timer registers.
3. Updates the software overflow counter.
4. Checks whether the required timing interval has elapsed.
5. Updates the real-time clock.
6. Checks the stopwatch active flag.
7. Updates the stopwatch when it is enabled.
8. Restarts the timer and returns from the interrupt.

This interrupt-driven architecture allows the normal clock to continue updating even while the stopwatch mode is selected.

8 Time Increment and Rollover Logic

Separate subroutines were implemented to update the real-time clock and stopwatch timing variables. The timing values were maintained in hours, minutes, and seconds.

The seconds value is incremented first. When the seconds count reaches its maximum value, it rolls over and increments the minutes count. Similarly, minute overflow increments the hours count, while the hours value follows a 24-hour rollover.

The general rollover sequence is:

$$\text{Seconds} \xrightarrow{60} \text{Minutes} \xrightarrow{60} \text{Hours} \xrightarrow{24} 0$$

The implementation uses arithmetic operations to determine the quotient and remainder during rollover. The remainder is stored as the updated lower-order time value, while an overflow causes the next higher-order timing value to be incremented.

The stopwatch follows the same incrementing structure but is updated only when the stopwatch active flag is enabled.

9 Clock and Stopwatch Operation

9.1 Clock Mode

The default operating mode displays the real-time clock in HH-MM-SS format. The timer interrupt routine periodically updates the stored clock values, while the display routine continuously refreshes the corresponding six digits.

The clock update sequence can be represented as:

$$\text{Timer Event} \rightarrow \text{Increment Clock} \rightarrow \text{Update Memory} \rightarrow \text{Refresh Display}$$

9.2 Stopwatch Mode

When stopwatch mode is selected, the display routine loads the stopwatch timing values instead of the normal clock values.

The stopwatch operation is controlled through a software active flag. When the flag is enabled, the timer interrupt routine calls the stopwatch increment subroutine. When the flag is disabled, the stopwatch values remain unchanged.

A separate reset routine is implemented to clear the stopwatch hours, minutes, and seconds values before beginning a new timing operation.

The stopwatch operation can therefore be summarized as:

$$\text{Select Stopwatch Mode} \rightarrow \text{Enable Stopwatch} \rightarrow \text{Timer-driven Increment} \rightarrow \text{Disable Stopwatch}$$

An important feature of the implementation is that the normal clock continues to update independently while the stopwatch operation is active.

10 Program Flow

The main program performs hardware initialization, configures the timer and interrupt system, reads the control inputs, and continuously updates the display.

The overall program flow is illustrated in Figure 2.

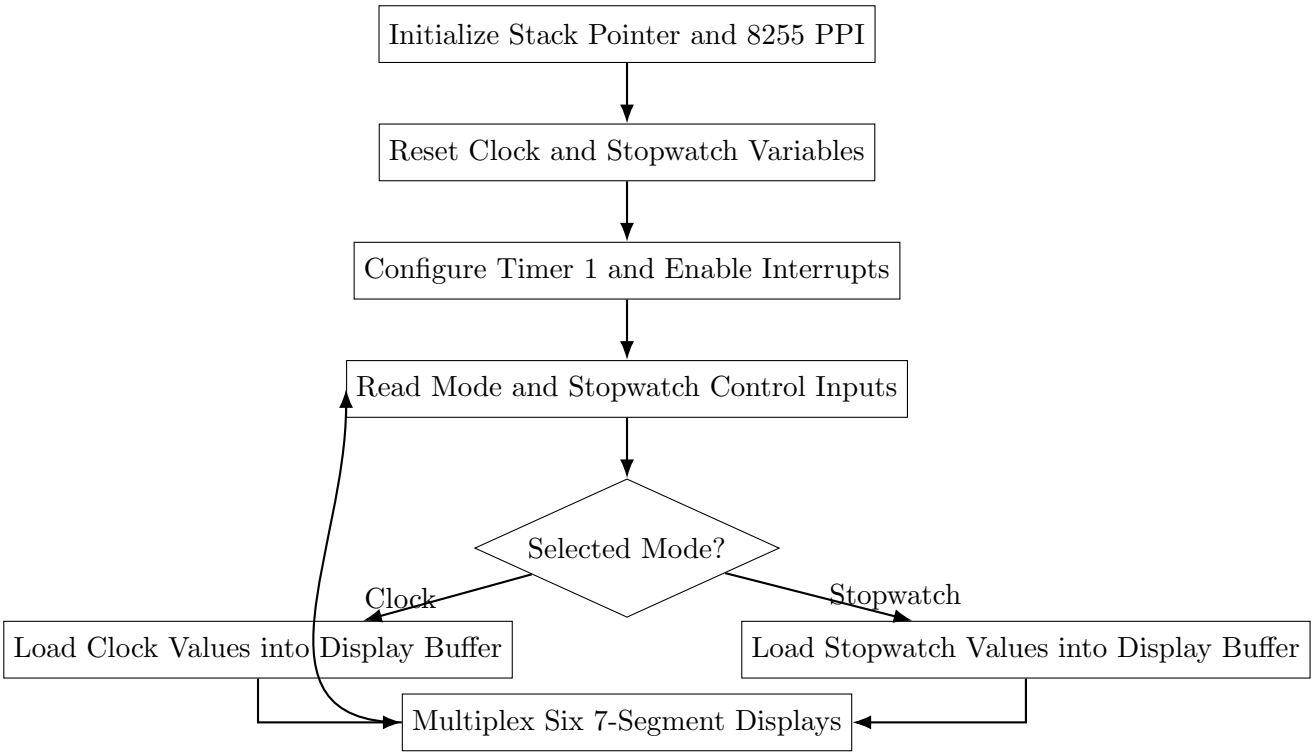


Figure 2: Main program flow of the implemented clock and stopwatch system.

11 Implementation Results and Discussion

The developed system demonstrated the integration of assembly-level microcontroller programming, timer-based interrupt handling, peripheral interfacing, and multiplexed display operation.

Timer 1 was used to generate periodic interrupt events, while a software counter was used to generate the required timing interval for updating the clock. The interrupt-driven approach allowed the main program to remain available for processing control inputs and refreshing the display.

The display driver successfully converted the stored hours, minutes, and seconds values into individual decimal digits. These digits were subsequently converted into the corresponding 7-segment patterns and displayed using multiplexed operation.

The major functions implemented in the system are summarized in Table 4.

Table 4: Functional implementation summary.

Function	**Implementation**
8051 Assembly Programming	Developed and executed on target hardware
Timer Configuration	Timer 1 configured for periodic interrupts
Interrupt Service Routine	Performed timer-based time updates
Clock Operation	Maintained hours, minutes, and seconds
Stopwatch Operation	Updated using independent timing variables
7-Segment Display	Six-digit multiplexed time display
8255 Interface	Connected displays and control inputs
Mode Selection	Switched between clock and stopwatch display
Stopwatch Control	Enabled and disabled through control input

The project provided practical experience with several important embedded-system concepts, including low-level assembly programming, external memory access, timer configuration, interrupt service routines, programmable peripheral interfaces, display multiplexing, and real-time control.

12 Conclusion

A real-time clock and stopwatch system was implemented using an 8051 microcontroller and 8051 assembly language. The program was developed in Keil uVision and programmed onto the target microcontroller kit using WIN51E.

The system uses the 8255 Programmable Peripheral Interface to connect common-cathode 7-segment displays and control switches. Six display digits are refreshed using multiplexing to display time in HH-MM-SS format.

Timer 1 and an Interrupt Service Routine were used to implement periodic timekeeping. Separate memory locations and increment routines were maintained for the real-time clock and stopwatch, allowing both timing systems to operate independently.

The project demonstrates the practical application of fundamental embedded system concepts including assembly-level programming, peripheral interfacing, timer interrupts, interrupt service routines, external memory organization, and hardware-software integration.